

一、渲染管线应用阶段 00:03

1. 知识回顾 00:10



唐老师系列教程-渲染管线 应用阶段



知识回顾

渲染管线（流水线）就是将数据分阶段的变为屏幕图像的过程

其中

数据就是我们在游戏场景中放置的模型、光源、摄像机等等内容的数据

阶段就是渲染管线中的三个阶段

应用阶段——>几何阶段——>光栅化阶段

通过这三个阶段对数据的处理，最终我们就能够在屏幕上看见图像

-
- 定义：渲染管线（流水线）是将数据分阶段变为屏幕图像的过程
- 数据来源：游戏场景中的模型、光源、摄像机等数据
- 三阶段划分：
 - 应用阶段（CPU主导）
 - 几何阶段
 - 光栅化阶段

2. 知识必备 00:47

1) CPU和GPU 00:57



唐老师系列教程-渲染管线 应用阶段



知识必备——CPU和GPU

CPU：中央处理器，负责算术运算、逻辑操作、数据传输等通用计算任务，同时还管理和调度计算机的资源（游戏中——游戏逻辑处理）

GPU：图形处理器，是专门用于图形和并行计算的处理器

显卡就是搭载GPU的硬件设备，显卡包含一个或多个GPU芯片，还包含显存（用于存储图像数据）、显示接口、视频解码器等等（游戏中——渲染相关处理）

CPU主要处理操作系统管理、程序执行、通用计算等等

GPU主要处理图形渲染、图像处理等等

-
- CPU核心功能：
 - 算术运算、逻辑操作、数据传输等通用计算
 - 计算机资源管理和调度
 - 游戏中处理游戏逻辑（角色移动、动画切换、寻路算法等）
- GPU核心功能：
 - 专门用于图形和并行计算的处理器
 - 显卡硬件包含GPU芯片、显存、显示接口等组件
 - 游戏中处理所有渲染相关任务
- 关键区别：
 - CPU：操作系统管理、程序执行、通用计算
 - GPU：图形渲染、图像处理等专用计算

2) 应用阶段主要做什么 02:47



唐老师系列教程-渲染管线 应用阶段



渲染管线 在应用阶段主要做什么



渲染管线的应用阶段中大部分的内容都和渲染无关（比如：游戏逻辑处理、动画更新、物理模拟、场景管理等）
当应用阶段完成后，后面的几何阶段以及光栅化阶段将开始处理和图形渲染相关的数据和操作。
那么应用阶段为什么会归到渲染管线中呢？
那是因为应用阶段为渲染管线的后续提供了最重要的内容——数据
应用阶段主导者是CPU，在这一阶段，我们将渲染需要用的数据传递给GPU用于后续的两个阶段的处理

WELCOME
TO THE
SPECIALTY COURSE
STUDIO
版权所有：唐老师 tpandme@163.com

- 主要内容：
 - 大部分工作与渲染无关（游戏逻辑、动画更新、物理模拟等）
 - 为后续阶段准备渲染数据
 - 纳入管线原因：
 - 提供渲染管线最基础的数据来源
 - CPU主导的数据准备阶段
 - 硬件分工：
 - 应用阶段在CPU执行
 - 准备完成后通过DrawCall通知GPU开始渲染
- 3) 应用阶段为渲染准备了些什么 04:06



唐老师系列教程-渲染管线 应用阶段



应用阶段为渲染准备了些什么？



- 1.把不可见的物体数据删除
- 2.准备好模型相关数据（顶点、法线、切线、贴图、着色器等）
- 3.将数据加载到显存中
- 4.设置渲染状态（设置网格需要使用那个着色器、材质、光源属性等等）
- 5.调用DrawCall（CPU通知GPU使用相关的数据和渲染状态进行渲染）

WELCOME
TO THE
SPECIALTY COURSE
STUDIO
版权所有：唐老师 tpandme@163.com

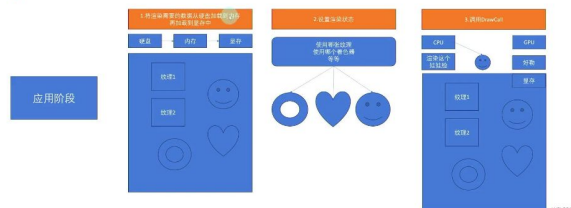
- 五大准备工作：
 - 可见性剔除：移除摄像机视野外的物体数据
 - 模型数据准备：收集顶点、法线、切线、贴图、着色器等数据
 - 显存加载：将数据从内存加载到显存
 - 状态设置：配置着色器、材质、光源等渲染参数
 - DrawCall调用：CPU向GPU发送渲染指令



唐老师系列教程-渲染管线 应用阶段



应用阶段为渲染准备了些什么？



WELCOME
TO THE
SPECIALTY COURSE
STUDIO
版权所有：唐老师 tpandme@163.com

- 数据流转路径：
 - 硬盘 → 内存（通过Resources.Load/AB包加载）
 - 内存 → 显存（网格信息、纹理等渲染数据）

- 最终由GPU处理显存中的数据

4) 应用阶段的内容总结 07:22



唐老狮系列教程-渲染管线 应用阶段



应用阶段为渲染准备了些什么？

渲染管线（流水线）中的**应用阶段**

主要是**CPU主导的阶段**

它为渲染完成的最主要的工作就是提供后续的渲染数据

比如：

顶点、法线、切线、纹理坐标、变换矩阵、材质属性等等

在应用阶段中，我们主要就是按照Unity的规则进行游戏开发即可

我们需要注意的就是关于DrawCall的优化

-
- 本质工作：数据准备与传递
- 关键数据：
 - 顶点数据（法线、切线、纹理坐标）
 - 变换矩阵
 - 材质属性
- 开发注意：
 - 遵循Unity开发规范
 - 重点优化DrawCall数量

WELCOME TO THE SPECIALTY COURSE STUDY
版权所有：唐老狮 tpandme@163.com

5) 关于DrawCall 08:09



唐老狮系列教程-渲染管线 应用阶段



关于DrawCall



我们之前在讲解UI相关知识时，提到了DrawCall，也知道它会影响我们游戏的性能

一次DrawCall其实就是CPU命令GPU进行渲染的命令

为什么DrawCall多了会影响性能呢？

主要的性能瓶颈是CPU造成的

每次调用DrawCall之前，CPU需要向GPU发送很多内容，包括数据、状态、命令等等。

如果DrawCall过多，CPU就会把大量的时间花费在提交DrawCall上，造成CPU过载，让玩家感受到卡顿

-
- 本质：CPU命令GPU渲染的指令
- 性能瓶颈：
 - 每次调用需要发送数据、状态、命令等大量信息
 - 过多调用会导致CPU时间消耗在准备工作上
 - 最终表现为游戏卡顿

WELCOME TO THE SPECIALTY COURSE STUDY
版权所有：唐老狮 tpandme@163.com

6) 如何减少DrawCall 09:08



唐老狮系列教程-渲染管线 应用阶段



如何减少DrawCall



使用批处理，可以有效的减少DrawCall，从而提升性能表现

1.合并网格（可以将静态物体合并网格）

2.共用材质（在不同网格之间共用一种材质）

3.合并图集（2D游戏和UI中，可以将多张图片合并为一张大图）

等等

-
- 批处理技术：

WELCOME TO THE SPECIALTY COURSE STUDY
版权所有：唐老狮 tpandme@163.com

- **网格合并**：静态物体合并为单一网格
- **材质共享**：不同网格使用相同材质
- **图集合并**：多张图片合并为一张大图（UI/2D游戏常用）

7) 内容总结 10:13



唐老师系列教程-渲染管线 应用阶段



总结

渲染管线（流水线）中的**应用阶段**

主要是**CPU主导的阶段**

它为渲染完成的最主要的工作就是提供后续的**渲染数据**

比如：

顶点、法线、切线、纹理坐标、变换矩阵、材质属性等等

在应用阶段中，我们主要就是按照Unity的规则进行游戏开发即可

DrawCall过多时，性能瓶颈是由CPU造成的，我们可以用批处理技术优化它

WELCOME
TO THE
SPECIALTY COURSE
5/1/2024
版权所有：唐老师 tpandme@163.com

-
- **阶段定位**：CPU主导的数据准备阶段
- **核心价值**：为后续渲染提供基础数据
- **优化重点**：通过批处理技术减少DrawCall
- **典型数据**：
 - 顶点属性（法线/切线/纹理坐标）
 - 变换矩阵
 - 材质参数

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
渲染管线基本概念	将数据分阶段变为屏幕图像的过程，包含 应用阶段 、 几何阶段 、 光栅化阶段	区分三个阶段的核心任务（数据准备 vs 图形处理 vs 像素生成）	★★
CPU与GPU功能对比	CPU ：通用计算（游戏逻辑、物理模拟）； GPU ：图形渲染（并行计算、着色器处理）	硬件选择标准：CPU影响逻辑性能，GPU决定渲染效率	★★★
应用阶段核心任务	1. 剔除不可见物体 2. 准备模型数据（顶点/法线/纹理） 3. 加载数据到显存	数据准备是核心 ，与渲染无关的逻辑（如动画更新）也在此阶段处理	★★★★

	4. 设置渲染状态（着色器/材质） 5. 调用 Draw Call		
Draw Call优化	性能瓶颈： CPU提交数据/命令耗时 优化方法： - 合并网格（静态批处理） - 共用材质 - 纹理图集（UI/2D游戏）	高频Draw Call导致 CPU过载 ，需理解批处理技术原理	★★★★
显存与内存交互	资源加载路径：硬盘 → 内存 → 显存 （网格/纹理数据）	Unity中Resources.Load或AB包加载仅到内存，渲染前需转移至显存	★★★